Autonomous Driving Using Double Deep Q-Networks with a Dueling Network Architecture

Vo Thien Phu
SE194286
FPT University
Ho Chi Minh City, Vietnam
phuvo05kid@gmail.com

Nguyen Truong Phuc Thinh
SE190168
FPT University
Ho Chi Minh City, Vietnam
nguyentruongphucthinh@outlook.com

***Abstract*—Reinforcement learning (RL) is well suited to sequential decision-making problems such as autonomous driving, but classical tabular methods do not scale to high-dimensional visual observations, and the Deep Q-Network (DQN) can suffer from overestimation bias. This paper investigates a Double Deep Q-Network (Double DQN) combined with a dueling network architecture in the Gymnasium CarRacing-v3 environment. The modular framework uses grayscale image preprocessing, frame stacking, action repetition, experience replay, target-network synchronization, ε-greedy exploration, and convolutional function approximation. Tabular Q-learning and vanilla DQN are implemented as baselines under a common protocol. After 1,000 training episodes, the proposed agent obtained a final episode reward of 820.59 and a final moving-average reward of 790.17 (training-time rewards); vanilla DQN obtained 846.70 and 819.47, and tabular Q-learning −8.48 and −19.40. The Double-DQN moving average plateaus near 775–800 after about episode 400. Both deep methods substantially outperformed the tabular baseline, while the single-run evaluation limits conclusions about the relative performance of the two deep methods. In an extension with physical obstacles, a speed-aware reward term and a larger replay memory raised the greedy policy's mean track progress from 44% to 64% (30 unseen tracks), although most episodes still contain a collision.**

***Index Terms*—autonomous driving, CarRacing-v3, deep reinforcement learning, Deep Q-Network, Double DQN, dueling network architecture**

I. Introduction

Autonomous driving requires an agent to perceive a changing environment, choose control actions, and account for the long-term consequences of those actions. Rule-based controllers can perform well in anticipated situations, but their behavior depends on manually specified rules and may not generalize to scenarios omitted during design. Reinforcement learning offers an alternative in which a policy is improved through interaction with an environment and feedback in the form of rewards.

Deep reinforcement learning extends this approach to high-dimensional sensory inputs. In particular, DQN demonstrated that a convolutional neural network can approximate action values directly from images [1]. Subsequent work introduced Double DQN to reduce the overestimation associated with maximization in the DQN target [2], and the dueling architecture to estimate state values and action advantages in separate streams [3].

This study applies these ideas to Gymnasium's CarRacing-v3 environment. Each episode contains a procedurally generated track, so the agent must learn a reusable driving policy rather than memorize a fixed route. The original visual observation and continuous controls are transformed into stacked grayscale frames and a discrete action set suitable for value-based learning.

The study addresses three questions: (1) whether Double DQN with a dueling architecture can learn an effective CarRacing-v3 policy; (2) how its observed performance compares with tabular Q-learning and vanilla DQN under the same protocol; and (3) whether its training behavior remains numerically stable. The contributions are a modular image-based RL framework, an agent that combines the Double-DQN target with a dueling head, and an empirical comparison with two value-based baselines in a single-run protocol. The scope is limited to a single simulated agent and does not include real vehicles, multi-agent learning, imitation learning, or continuous-action actor–critic methods.

II. Background

A. Reinforcement Learning and Markov Decision Processes

A reinforcement-learning problem is commonly represented as a Markov decision process (MDP) $M = (S, A, P, R, \gamma)$, where S is the state space, A the action space, P the transition model, R the reward function, and $0 \le \gamma < 1$ the discount factor. At time t, the agent observes s_t, executes a_t, receives r_t, and observes s_{t+1}. Its objective is to maximize the discounted return

$$G_t = \sum_{k=0}^{\infty} \gamma^k \, r_{t+k+1} \tag{1}$$

The action-value function $Q(s,a) = \mathrm{E}[G_t \mid s_t = s,\ a_t = a]$ estimates the long-term value of an action, and a greedy policy selects $\arg\max_a Q(s,a)$.

B. Q-Learning and DQN

Tabular Q-learning updates a stored state–action value after observing a transition:

$$Q(s,a) \leftarrow Q(s,a) + \alpha\left[r + \gamma \max_{a'} Q(s',a') - Q(s,a)\right] \tag{2}$$

where α is the learning rate. This representation is impractical for raw images because the number of possible observations is

enormous and nearby visual states do not share estimates. DQN replaces the table with a neural approximation $Q(s, a; \theta)$ [1]. For visual states, convolutional layers learn features jointly with the value function. Experience replay reduces temporal correlation by sampling stored transitions, while a periodically updated target network supplies more slowly changing learning targets. The standard DQN target is

$$y^{\mathrm{DQN}} = r + \gamma \max_{a'} Q_{\mathrm{target}}(s', a') \tag{3}$$

Because maximization and evaluation use the same estimates, noise can produce optimistic targets.

C. Double DQN and Dueling Value Estimation

Double DQN separates action selection and evaluation [2]. The online network selects

$$a^{*} = \arg\max_{a} Q_{\mathrm{online}}(s', a) \tag{4}$$

and the target network evaluates that action:

$$y^{\mathrm{DDQN}} = r + \gamma\, Q_{\mathrm{target}}(s', a^{*}) \tag{5}$$

For terminal transitions, the bootstrapped term in (5) is omitted. The dueling architecture divides shared features into a scalar state-value stream $V(s)$ and an action-advantage stream $A(s, a)$ [3]. Their identifiable aggregation is

$$Q(s, a) = V(s) + A(s, a) - \frac{1}{|\mathcal{A}|} \sum_{a' \in \mathcal{A}} A(s, a') \tag{6}$$

This decomposition can help the network learn the quality of a state even where several actions have similar consequences.

III. Related Work

Mnih *et al.* established DQN as a practical method for learning control policies from pixels by combining convolutional function approximation, replay memory, and a target network [1]. Their Atari experiments motivated the common visual-control pipeline adopted here.

Van Hasselt *et al.* showed that DQN can substantially overestimate action values and proposed the Double-DQN target, which uses the online network for selection and the target network for evaluation [2]. Wang *et al.* subsequently proposed the dueling architecture, demonstrating that separate value and advantage streams can be combined with existing DQN improvements [3]. Rainbow [5] later combined these and other extensions (prioritized replay, multi-step returns, distributional and noisy components) in one agent. The present work integrates only the Double-DQN target and the dueling architecture for CarRacing-v3 and compares the resulting agent with tabular Q-learning and vanilla DQN in one software framework.

CarRacing is a common benchmark for pixel-based control. Ha and Schmidhuber trained a controller on a learned world-model representation of this task [6], and policy-gradient and actor–critic methods such as PPO [7] and SAC [8] are commonly used with its native continuous action space. Value-based methods, as used here, require discretizing the controls (Section IV-C).

IV. Methodology

A. Environment and Problem Formulation

CarRacing-v3 [4] supplies a 96 × 96 RGB observation. In its native form it accepts a continuous vector of steering in [−1, 1], throttle in [0, 1], and braking in [0, 1]; the environment returns −0.1 per frame and +1000/N for each of the N track tiles visited for the first time. A transition stored for deep learning is

$$(s_t, a_t, r_t, s_{t+1}, d_t) \tag{7}$$

where d_t indicates termination. The policy network predicts one value per discrete action, and exploitation chooses the action with the largest predicted value.

B. Observation Preprocessing and State Representation

Each RGB frame is converted to grayscale, resized to 84 × 84 with area interpolation, scaled to [0, 1], and appended to a rolling stack of four frames. The resulting state has shape 4 × 84 × 84. Frame stacking supplies short-term motion cues that a single image cannot provide and approximates otherwise hidden variables such as velocity and turning direction. The same preprocessing is used for the vanilla-DQN and Double-DQN agents to avoid an input-related confound.

C. Discrete Action Space

The agent uses the built-in discrete mode of CarRacing-v3 (`continuous=False`), which exposes five actions with the control values given in Table I. Each action sets exactly one control: steering, throttle and braking are never combined.

TABLE I
DISCRETE ACTION MAPPING EXECUTED BY CARRACING-V3

Index	Action	Steering	Throttle	Brake
0	Idle	0.0	0.0	0.0
1	Steer left	−0.6	0.0	0.0
2	Steer right	+0.6	0.0	0.0
3	Accelerate	0.0	0.2	0.0
4	Brake	0.0	0.0	0.8

D. Action Repetition, Reward Handling and Early Termination

Each selected action is repeated for four simulator frames and the four frame rewards are summed into a single step reward; only the last frame is added to the stack. Rewards are neither clipped nor normalized. The wrapper also ends an episode early when the summed step reward is negative for more than 25 consecutive agent steps (i.e., no new tile is visited for roughly 100 frames) and adds an extra penalty of −50 at that moment. The reported returns therefore include this penalty and are not directly comparable with scores obtained from the unmodified environment. The maximum episode length is the 1,000-frame limit of CarRacing-v3, i.e., 250 agent steps. Gymnasium reports natural termination and this time-limit truncation as separate flags; the wrapper merges them, together with the early-stop rule, into a single done flag, so the stored transition has $d = 1$ in all three cases and the bootstrap term in (5) is omitted at time-limit and early-stop boundaries as well. Standard practice is to keep bootstrapping at truncation; the effect

of this choice on the results was not measured.

E. Proposed Network

The online and target networks have identical architectures. Each accepts the four-frame state and uses three convolutional layers (32 filters of 8×8 with stride 4, 64 of 4×4 with stride 2, and 64 of 3×3 with stride 1, each followed by ReLU) to produce a 64×7×7 feature map. The flattened 3,136-dimensional vector branches into two fully connected streams (3,136→512→1 for V and 3,136→512→5 for A) and is recombined using (6). The online network selects actions and receives gradient updates; the target network is synchronized only at fixed intervals and supplies the evaluation term in (5). The complete architecture is shown in Fig. 1.

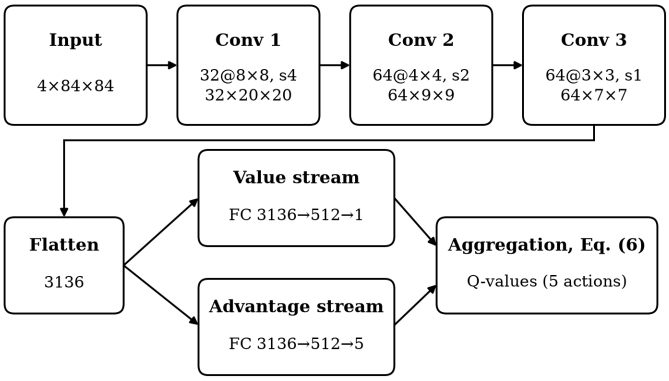


Fig. 1. Proposed Double DQN with a dueling network architecture (each convolution is followed by ReLU).

The online network minimizes the Huber (smooth L1) loss on the temporal-difference error between predicted values and Double-DQN targets. Adam performs optimization, gradient-norm clipping at 10 limits unusually large updates, and periodic target synchronization provides a stable reference. Because rewards are not normalized, the Q-values, and hence the TD errors, can reach large magnitudes (returns of several hundred, plus the −50 early-stop penalty), which is relevant to the loss behavior discussed in Section VII.

F. Training Procedure

Action selection follows an ε-greedy policy: a random action is selected with probability ε, and the greedy network action is selected otherwise. In the implementation ε is multiplied by 0.9999 after *every optimization step* (not every episode) until it reaches 0.05, which takes about 29,960 updates. Transitions are stored in a replay memory of 10,000 entries; once at least 32 transitions are available, one mini-batch update is performed after every agent step, with uniform sampling. Algorithm 1 summarizes the procedure.

Algorithm 1 Double-DQN Training Procedure

Initialize online network $Q(\cdot;\theta)$, target network $Q(\cdot;\theta^{-})$, replay memory D
Set $\theta^{-} \leftarrow \theta$
for each training episode **do**
  Reset the environment; construct the initial stacked state s
  while the episode is not finished **do**
    Random action with probability ε; else $\arg\max_a Q(s, a; \theta)$
    Repeat the action for 4 frames; observe (r, s', d)
    Store (s, a, r, s', d) in D
    if $|D| \geq 32$ **then**
      Sample a mini-batch uniformly from D
      Compute Double-DQN targets using (5)
      Update θ (Huber loss, Adam, gradient clipping)
      $\varepsilon \leftarrow \max(0.05, 0.9999 \cdot \varepsilon)$
      Every 1,000 updates set $\theta^{-} \leftarrow \theta$
    end if; $s \leftarrow s'$
  end while; save logs/checkpoints as scheduled
end for

V. Implementation

The system uses a modular architecture that separates environment management, observation preprocessing, agents, neural networks, replay memory, logging, checkpoints, training, and evaluation. The environment wrapper converts simulator observations so that the learning algorithms operate only on stacked tensors and discrete indices. The agent class encapsulates action selection, transition storage, optimization, target synchronization, persistence, and recovery.

Training and inference use separate entry workflows. The training workflow constructs the environment and agent, performs interaction and optimization, records rewards, losses, and exploration rates, and periodically saves model and optimizer state; it also saves a checkpoint on interruption and a "best" checkpoint whenever the 100-episode average improves after episode 20. The evaluation workflow loads a checkpoint, disables learning and random exploration, and runs five episodes on fixed seeds (42, 123, 777, 999, 2024). The proposed agent and both baselines share the preprocessing, action definitions, logging, and evaluation procedure wherever applicable. A Tkinter demo application replays a saved checkpoint with a live view of reward, chosen action, speed, collisions and a minimap, and also allows keyboard driving. The code, the Double-DQN training log, this report, the obstacle-challenge environment of Section VIII, Colab/Kaggle training notebooks and a demo video are available at https://github.com/phuvo05/vroom; of the learning agents, the repository contains the Double-DQN agent only.

VI. Experimental Setup

A. Training Configuration and Baselines

The environment uses 84×84 images, four stacked frames, and four-frame action repetition. All agents were trained for 1,000 episodes. Table II reports the configuration of the deep agents as defined in `config.py`: a replay capacity of 10,000 transitions, mini-batches of 32, and target-network synchronization every 1,000 optimization steps.

The first baseline is tabular Q-learning, for which the visual state must be discretized before a Q-table can be used. The second is vanilla DQN, which uses convolutional approximation, replay memory, and a target network but retains the standard maximization target. The proposed agent adds Double-DQN target computation and the dueling value decomposition. Shared processing and evaluation components are held constant to improve comparability. The state discretization and Q-table size of the tabular baseline, its learning rate and exploration schedule, and whether vanilla DQN shares the convolutional trunk of the proposed network with a single output head are not documented in the released repository (Section VII-D).

TABLE II
TRAINING HYPERPARAMETERS

Hyperparameter	Value
Input image size	84 × 84
Frame-stack depth	4
Action-repeat frames	4
State dimensions	4 × 84 × 84
Number of actions	5
Discount factor γ	0.99
Learning rate (Adam)	1×10^{-4}
Mini-batch size	32
Replay-buffer capacity	10,000
Target-update interval	1,000 updates
Initial ε	1.00
Minimum ε	0.05
ε decay factor (per update)	0.9999
Gradient-norm clip	10
Early-stop rule	>25 negative steps, −50
Training episodes	1,000
Checkpoint interval	50 episodes

B. Metrics and Protocol

Episode reward is the principal policy-performance metric. The logs also record a 100-episode moving-average reward, the mean Double-DQN optimization loss of the episode, and the current exploration rate (Table III). **All reward figures in this paper are training-time returns collected with ε-greedy exploration, not greedy-policy evaluation returns.** For Double DQN, ε stays at its floor of 0.05 after episode 219; the baselines reach their floor at other episodes (Figs. 2–3). Each algorithm was trained independently with the common environment configuration. Only one training run per algorithm is reported; consequently, the experiment describes observed runs rather than estimating a performance distribution across random seeds. The training seed, hardware and wall-clock time of these runs were not recorded.

TABLE III
EVALUATION METRICS

Metric	Description
Episode	Sequential training-episode index.
Reward	Cumulative reward obtained during the episode (including early-stop penalty).
Average reward	Moving mean of episode rewards over up to 100 episodes.
Loss	Mean Huber loss over the optimization steps of the episode.
Exploration rate	Current ε of the ε-greedy policy.

VII. Results And Discussion

A. Training Performance

The training histories are presented separately for clarity. The tabular Q-learning curves in Fig. 2 show limited reward improvement and a final episode reward of −8.48 with a 100-episode average of −19.40. The vanilla-DQN curves in Fig. 3 show a transition from negative early rewards to a final episode reward of 846.70 and a 100-episode average of 819.47. The proposed Double-DQN curves in Fig. 4 similarly show substantial reward growth, ending with a final episode reward of 820.59 and a 100-episode average of 790.17. All three exploration schedules reach the minimum ε of 0.05, while Fig. 4 additionally reports the Double-DQN training loss. Table IV gives the numerical comparison from episode 1,000 of the three training logs.

A closer reading of the Double-DQN log (Table V) shows three phases. During the first 100 episodes (ε falling from 1.0 to 0.40) episodes end with returns close to the −50 early-stop penalty, with a mean of −42.6. The 100-episode average first exceeds 0 at episode 148, 300 at episode 221, 500 at episode 258, and 700 at episode 364; ε reaches its floor at episode 219. From episode 400 onward the return plateaus: window means stay between 766 and 778 and the best 100-episode average, 808.24, occurs at episode 976. The best single episode (926.90) occurs at episode 322, and the final window is not better than episodes 401–600, so additional training after about episode 400 produced no clear gain in this run.

TABLE IV
FINAL TRAINING PERFORMANCE AFTER 1,000 EPISODES

Algorithm	Final reward	100-ep. average
Tabular Q-learning	−8.48	−19.40
Vanilla DQN	846.70	819.47
Double DQN (proposed)	820.59	790.17

TABLE V
DOUBLE-DQN TRAINING LOG BY EPISODE WINDOW

Episodes	Mean	Std	Min	Max	Loss
1–100	−42.6	14.6	−70.4	−1.8	0.76
101–200	180.3	205.9	−50.7	744.8	1.16
201–400	686.0	173.9	21.3	926.9	2.26
401–600	777.9	138.6	66.9	918.0	3.60
601–800	766.3	153.2	91.0	922.8	4.16
801–1000	775.0	163.6	14.1	922.7	4.67

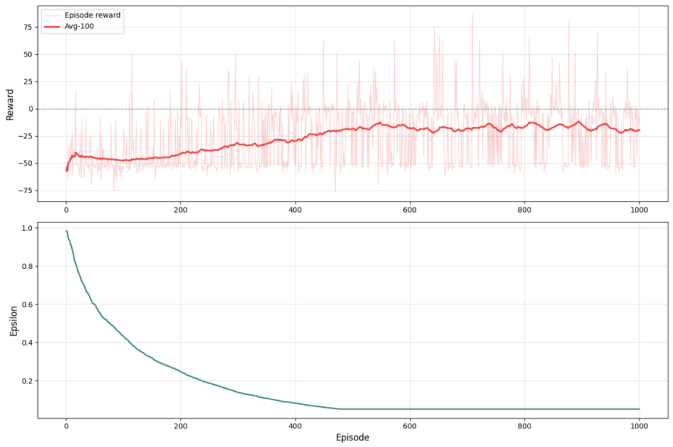


Fig. 2. Tabular Q-learning episode reward, 100-episode moving-average reward, and exploration rate over 1,000 episodes.

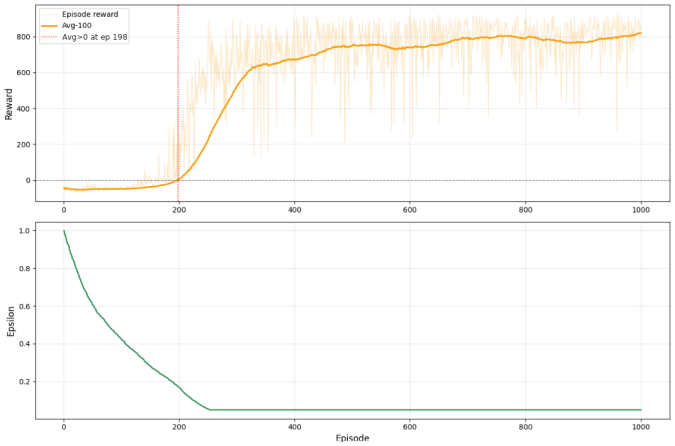


Fig. 3. Vanilla-DQN episode reward, 100-episode moving-average reward, and exploration rate over 1,000 episodes.

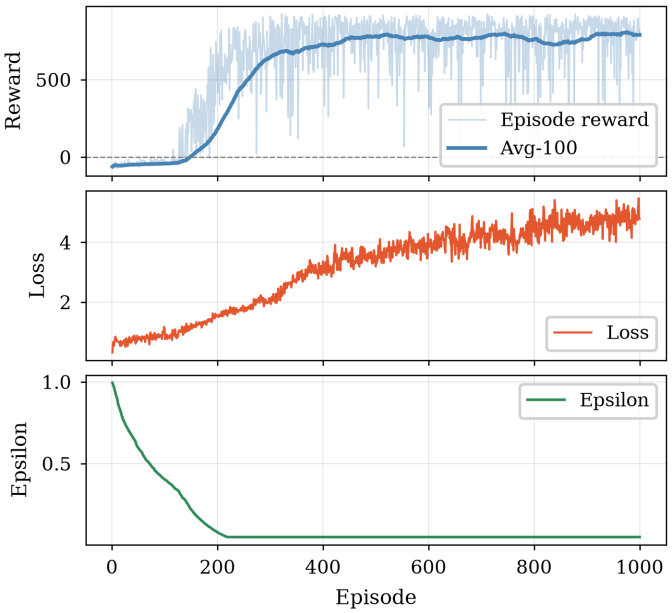


Fig. 4. Double-DQN reward, 100-episode moving-average reward, training loss, and exploration rate over 1,000 episodes.

B. Comparative Analysis

Both neural methods substantially outperformed tabular Q-learning in the reported runs. This difference is consistent with the representational demands of a visual state space: a neural model can share learned features across similar images, whereas a table treats discretized states independently and scales poorly as resolution increases.

Vanilla DQN produced the larger final and moving-average rewards in this experiment (819.47 vs. 790.17). Therefore, the current evidence does not support a claim that the proposed model improves reward over vanilla DQN. It does show that the proposed model learned a comparably effective policy while incorporating target and architectural mechanisms motivated by reduced overestimation and improved state-value representation. Within a single run, a 100-episode average has a standard error of roughly 16 (an episode-level standard deviation of about 156 divided by 10, the square root of the window size), but this does not capture the dominant source of uncertainty: variation between independent training runs caused by random initialization, exploration paths and replay sampling. With one run per algorithm that variation is unknown, so the 29-point gap between the two moving averages can neither be attributed to the method nor dismissed, and no ranking of the two deep methods is claimed.

C. Training Stability

The Double-DQN log shows decaying exploration to 0.05, an increasing moving-average reward, and no NaN or sudden loss explosion. However, the mean episode loss rises almost monotonically from 0.76 (episodes 1–100) to 4.67 (episodes 801–1000) and has not clearly flattened by episode 1,000 (Fig. 4). A rising loss is plausible here: the Huber loss acts on temporal-difference errors, and with unnormalized rewards the Q-values grow as the agent learns to collect tile rewards, so the TD errors grow with them. The −50 early-stop penalty and the treatment of truncation as termination (Section IV-D) can also create large TD errors, although their individual contributions were not isolated. The loss is therefore not a reliable indicator of convergence here. In the last 100 episodes about 6% of episodes fall below 500 (minimum 196.5), showing that occasional failures persist. These observations are compatible with usable but not fully stable behavior in the recorded run and should not be interpreted as a convergence guarantee. Multiple seeds, confidence intervals, greedy evaluation on standardized tracks, and direct measurements of value overestimation would be needed for a stronger comparison.

D. Reproducibility Check of the Released Code

To verify the released implementation, its dependencies were installed in a clean environment (Python 3.11.14, PyTorch 2.14.1+cu126, Gymnasium 1.4.0, box2d 2.3.10, NumPy 2.4.6, opencv-python 5.0.0.93) on Windows 11 with an NVIDIA GeForce GTX 1650 SUPER (4 GB). The training entry point was run for six episodes with its default configuration, writing logs and checkpoints to a separate directory. It completed without errors on the GPU, at 2.5–9.7 s per early episode, produced episode returns from −77.5 to −53.8 (100-episode average −61.4 after six episodes), and saved checkpoints at

episodes 3 and 6. These returns are of the same order as the first episodes of the reported log (mean −42.6 over episodes 1–100, minimum −70.4). A checkpoint was reloaded and evaluated greedily on two seeds, giving returns of 30.7 and −28.4, which only confirms that the load–evaluate path works for an almost untrained network.

This check does not reproduce the 1,000-episode result of Table IV, which would require several hours of training and, because of random initialization, would not yield identical numbers. Further limitations apply: the repository contains the code and log for the Double-DQN agent only, so the tabular Q-learning and vanilla-DQN rows and Figs. 2–3 could not be regenerated from it; no pre-trained checkpoint is included, so the greedy evaluation workflow of Section V was not run on the final model of the 1,000-episode run, whose results would complement the training-time returns reported here.

VIII. Extension: Obstacle Driving

A. Environment Extension

To test the agent beyond track following, the environment was extended with three scenery themes (forest, city, desert), physical obstacles and optional rain. Barriers, parked cars and crates are static Box2D bodies placed every 23 track tiles (13 in a hard mode), alternately left and right of the centerline at a lateral offset of 24–43% of the track width, so a free lane always remains. They appear in both the agent's observation and the demo view. The first contact with an obstacle costs −80 reward and adds 35% damage (50% in hard mode); 100% damage ends the episode. Rain lowers tire–road friction from 1.0 to 0.62, with puddles at 0.32. The classic/dry mode is unchanged, so the agent of Sections VI–VII remains comparable. Preprocessing and the action set of Section IV are kept; in particular, the car has no reverse gear.

B. Failure Analysis of the First Obstacle Agent

A first agent was trained in the forest scene at normal difficulty for about 3,000 episodes (415,482 optimization steps) with the configuration of Table II. Greedy evaluation on ten tracks (seeds 1000–1009, inside the range of training seeds) gave a mean track progress of 43.8%, 1.4 collisions per episode and one completed lap (Table VI). Logging the vehicle speed together with the heading change over the next ten track tiles on three tracks showed a mean speed of about 180–220 km/h even in sharp bends (heading change above 0.5 rad), and in the demo the agent missed such bends at high speed. After an obstacle contact it often stopped against the body, and the episode then ended through the early-stop rule of Section IV-D.

C. Speed-Aware Reward Shaping and Training Changes

To make cornering speed part of the objective, a shaping term is subtracted from the reward of every simulator frame in the obstacle scenes:

$$r' = r - 0.01 \max(0,\ v - v_{\text{safe}}) \tag{8}$$

$$v_{\text{safe}} = \text{clip}(220 - 110\,\kappa,\ \ 60,\ \ 220) \tag{9}$$

where v is the speed in km/h and κ the absolute heading change (rad) between the nearest tile and the tile ten positions ahead. At 220 km/h in a 1-rad bend the term costs 1.1 per frame (4.4 per agent step), comparable to the roughly 3.5 reward per step that the first agent collected at full speed.

Two further changes were made. The replay memory was enlarged from 10,000 to 100,000 transitions, with frames stored as 8-bit integers (lossless, because preprocessed pixels are multiples of 1/255), which keeps it near 5.6 GB. Training was warm-started from the first agent's weights with a fresh optimizer, an empty memory and ε reset to 0.5, because the reward had changed. The agent was trained for 1,500 episodes on a Kaggle GPU (261,067 optimization steps, about 3.9 h), and the checkpoint with the highest 100-episode training average was kept. That average was about 254–256 near episode 1,185 and 238 at episode 1,500, so the last 300 episodes brought no further gain.

D. Results

Table VI summarizes greedy evaluations. The shaped-reward agent was evaluated on ten unseen tracks on the training GPU and on 30 unseen tracks on a CPU; seed 100042, included in both, gave 81.6% and 89.9% progress, reflecting small numerical differences between devices. Mean progress rose from 43.8% to 61.6–64.5% and collisions per episode fell from 1.4 to 1.1. Six of the 30 tracks were driven to at least 90% and five without any collision. No episode completed a lap: the four best runs, used for the demo video, ended at the 250-step time limit at 90–98% of the track, consistent with the lower driving speed. Returns are not compared, because the two agents optimize different rewards. Fig. 5 shows the agent passing a parked car.

TABLE VI
GREEDY EVALUATION IN THE FOREST SCENE (NORMAL DIFFICULTY)

Agent and tracks	Progress	Collisions	Laps	No collision
First agent, seeds 1000–1009	43.8%	1.4	1/10	0/10
Shaped reward, seeds 100042–100051 (GPU)	61.6%	1.1	0/10	1/10
Shaped reward, seeds 100042–100071 (CPU)	64.5%	1.1	0/30	5/30

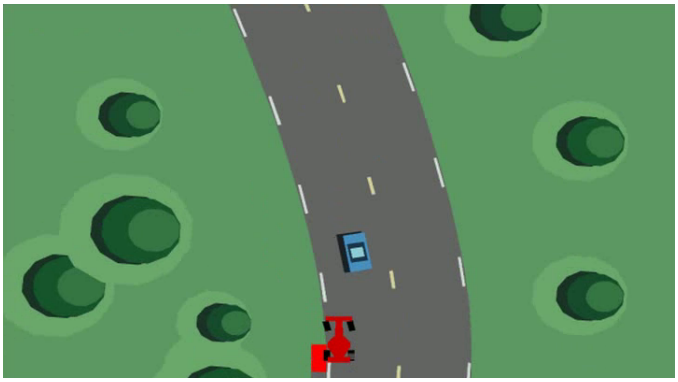


Fig. 5. Shaped-reward agent passing a parked car in the forest scene (greedy policy, seed 100043).

E. Limitations

The two agents were evaluated on different tracks, and those of the first agent lie inside its training-seed range, so the comparison is indicative rather than controlled; a paired evaluation on the same unseen tracks is still needed. Each result comes from a single training run in one scene and difficulty, and the shaping coefficients were chosen from the measurements above rather than tuned. Collisions remain the main failure: 25 of 30 tracks contain at least one, after which the agent may become stuck. Rewarding obstacle avoidance before contact, adding a reverse action, and training across scenes and difficulties are natural next steps.

IX. Conclusion And Future Work

This paper presented a modular reinforcement-learning framework for autonomous driving in CarRacing-v3. It transforms visual observations into stacks of four 84×84 grayscale frames, repeats a five-action discrete control set over four frames, and combines convolutional perception with Double-DQN targets and a dueling network architecture. Under the reported 1,000-episode protocol, both deep agents learned high-reward policies, while tabular Q-learning did not solve the image-based task. The proposed agent achieved a final reward of 820.59 and a final moving-average reward of 790.17, plateauing from about episode 400; vanilla DQN obtained slightly higher values in the single reported run. A smoke test of the released code confirmed that the pipeline installs and trains as documented. In an obstacle-driving extension, speed-aware reward shaping and a larger replay memory raised greedy track progress from 44% to about 64%, while collision avoidance remains unsolved.

The study is limited by evaluation in one simulator, one training run per algorithm, training-time rather than greedy evaluation returns, a modified reward (early termination with a −50 penalty), the treatment of time-limit truncation as termination, a steadily rising loss, undocumented baseline configurations, an unrecorded training seed, hardware and wall-clock time, and discrete rather than continuous control. Future work should report complete software, hardware, and hyperparameter configurations for every algorithm; release baseline code, logs and trained checkpoints; repeat experiments across random seeds with confidence intervals and statistical tests; evaluate greedy policies on held-out tracks with fixed seeds; add ablations (Double-DQN target only, dueling head only) to separate the effect of each component; bootstrap at time-limit truncation; and consider reward normalization or clipping. Further extensions could investigate prioritized replay, noisy networks and the full Rainbow combination [5], attention-based encoders, and continuous-control algorithms such as PPO [7], SAC [8], or DDPG, and evaluation in more realistic simulators.

References

[1] V. Mnih, K. Kavukcuoglu, D. Silver, *et al.*, "Human-level control through deep reinforcement learning," *Nature*, vol. 518, no. 7540, pp. 529–533, 2015, doi: 10.1038/nature14236.

[2] H. van Hasselt, A. Guez, and D. Silver, "Deep reinforcement learning with Double Q-learning," in *Proc. 30th AAAI Conf. Artificial Intelligence*, 2016, pp. 2094–2100, doi: 10.1609/aaai.v30i1.10295.

[3] Z. Wang, T. Schaul, M. Hessel, H. van Hasselt, M. Lanctot, and N. de Freitas, "Dueling network architectures for deep reinforcement learning," in *Proc. 33rd Int. Conf. Machine Learning*, PMLR vol. 48, 2016, pp. 1995–2003.

[4] M. Towers *et al.*, "Gymnasium: A standard interface for reinforcement learning environments," arXiv:2407.17032, 2024.

[5] M. Hessel *et al.*, "Rainbow: Combining improvements in deep reinforcement learning," in *Proc. 32nd AAAI Conf. Artificial Intelligence*, 2018, pp. 3215–3222, doi: 10.1609/aaai.v32i1.11796.

[6] D. Ha and J. Schmidhuber, "Recurrent world models facilitate policy evolution," in *Advances in Neural Information Processing Systems 31*, 2018.

[7] J. Schulman, F. Wolski, P. Dhariwal, A. Radford, and O. Klimov, "Proximal policy optimization algorithms," arXiv:1707.06347, 2017.

[8] T. Haarnoja, A. Zhou, P. Abbeel, and S. Levine, "Soft actor-critic: Off-policy maximum entropy deep reinforcement learning with a stochastic actor," in *Proc. 35th Int. Conf. Machine Learning*, PMLR vol. 80, 2018, pp. 1861–1870.